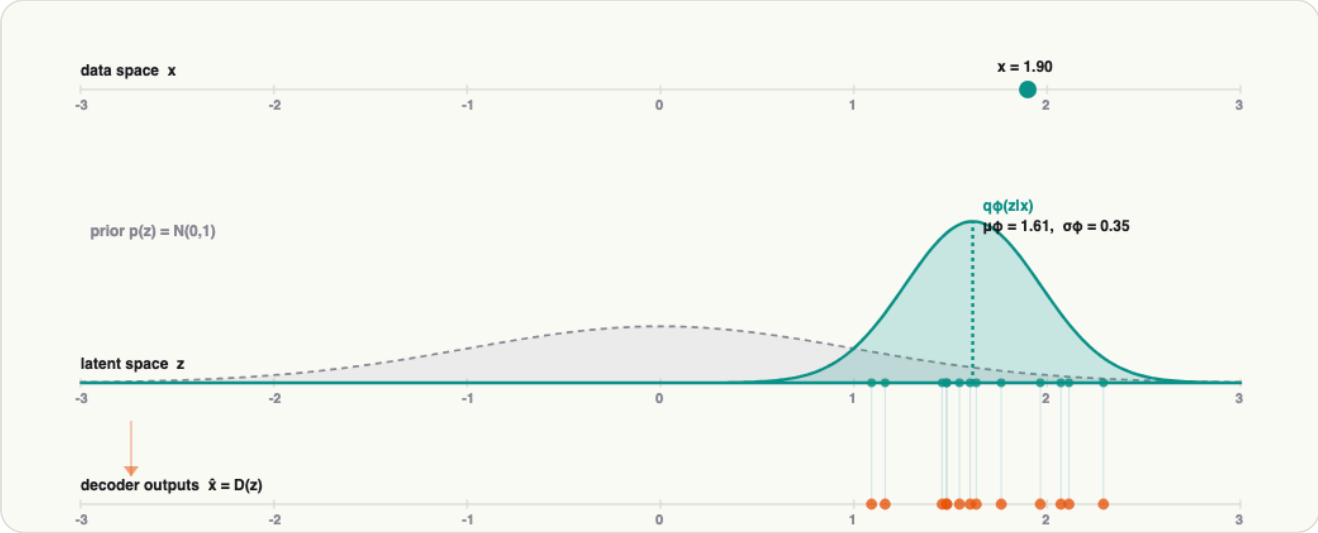
VAEs — Cheat Sheet & Visualizer

MLE → Jensen → ELBO → EM/GMM → Reparameterization → VQ-VAE

Latent-variable models from maximum likelihood to the reparameterization trick and posterior collapse.

Interactive — the ELBO trade-off & posterior collapse

Reconstruction wants a tight, informative $q_\phi(z|x)$. The KL term pulls it onto the prior $p(z) = \mathcal{N}(0, I)$. Push β too high and $\mu_\phi \to 0,\ \sigma_\phi \to 1$ for *every* x — the decoder can no longer tell inputs apart.



recon ≈ **0.204** · D_{KL} = **1.915** · recon+β·KL = **2.119** → **reconst. vs. regularity**

p43 Latent Variable Models

$$\text{Data } D = \{x_1, x_2, \ldots, x_n\} \sim \text{iid } P_x$$

Suppose P_θ denotes a parametric model.

Definition of a Latent variable Model

$$P_\theta(x) = \sum_z P_\theta(x, z) \quad (\text{if } z \text{ is discrete}) \qquad \text{or} \qquad P_\theta(x) = \int_z P_\theta(x, z)\, dz \quad (\text{if } z \text{ is continuous})$$

Z : **Latent / Hidden / unobserved R.v.**

Typically, z is jointly estimated along with the model parameters, θ. For each $x_i \in D$, we assume that there exists a z_i corresponding to each x_i.

Eg: **(a)** z is discrete $\in \{1, 2, \ldots, M\}$, $x \in \mathbb{R}^d$. For each $x_i \in D$, $z_i|x_i$ represents the latent var corresponding to x_i. Since z_i is discrete, each x_i is **clustered** into one of M categories: **clustering**. → Gaussian Mixture Models, K-means clustering.

p44 Continuous latents + the learning objective

(b) z : continuous. $x \in \mathbb{R}^d$, $z \in \mathbb{R}^k$, **Auto-Encoder**, $k \ll d$. Here, $z_i|x_i$ represent a **feature-vector** corresponding to the given x_i.

Latent variable Models can also be used as **generative models**.

General principle for Learning Latent var. Models

Suppose we are given Data $D = \{x_i\}_{i=1}^n \sim$ iid P_x, and $P_\theta(x) = \int_z P_\theta(x,z)\,dz$ be the lat var. Model. **Goal:** Estimate the model parameters θ, given D.

$$\theta^* = \arg\min_\theta \left[\, D_{KL}(P_x \| P_\theta) \,\right]$$

$$= \arg\min_\theta \left[\int_x P_x(x) \log \frac{P_x(x)}{P_\theta(x)} dx \right]$$

$$= \arg\min_\theta \left[\underbrace{\int_x P_x(x) \log P_x(x)\, dx}_{\text{Independent of } \theta} - \int_x P_x(x) \log P_\theta(x)\, dx \right]$$

p45 Maximum Likelihood Estimation

$$= \arg\min_\theta \left[- \int_x P_x(x) \log P_\theta(x)\, dx \right]$$

$$\boxed{\theta^* = \arg\max_\theta \left[\, \mathbb{E}_{P_x} \log P_\theta(x) \,\right]} \quad : \text{ maximum Likelihood Estimation}$$

$\log P_\theta(x)$: **Log Likelihood function** of $P_\theta(x)$.

$P_\theta(x)$ in a latent variable model: $P_\theta(x) = \int_z P_\theta(x,z)\,dz$

$$\theta^* = \arg\max_\theta \left[\mathbb{E}_{P_x} \log P_\theta(x) \right]$$

Let us denote $\log P_\theta(x)$ with $L(\theta)$.

$$L(\theta) = \log P_\theta(x) = \log \int_z P_\theta(x,z)\, dz$$

Suppose $q(z|x)$ denote a density function over the latent variable, z given x.

$$L(\theta) = \log \int_z P_\theta(x,z) \frac{q(z|x)}{q(z|x)}\, dz$$

p46 Jensen's inequality → ELBO

$$= \log \int_z q(z|x) \left[\frac{P_\theta(x,z)}{q(z|x)} \right] dz \qquad \because \int_z q(z|x) g(z)\, dz = \mathbb{E}_{q(z|x)} g(z)$$

$$L(\theta) = \log\ \mathbb{E}_{q(z|x)} \left(\frac{P_\theta(x,z)}{q(z|x)} \right)$$

Jensen's Inequality: $\log \mathbb{E}(\cdot)\ \geq\ \mathbb{E} \log(\cdot)$

$$L(\theta) = \log \mathbb{E}_{q(z|x)} \left(\frac{P_\theta(x,z)}{q(z|x)} \right)\ \geq\ \mathbb{E}_{q(z|x)} \log \left(\frac{P_\theta(x,z)}{q(z|x)} \right)$$

$$\text{denote the RHS with } J_\theta(q)$$

$$L(\theta)\ \geq\ J_\theta(q), \qquad \text{called the Evidence Lower Bound (ELBO)}$$

$$(\text{the "Evidence" is } L(\theta))$$

- $J_\theta(q)$ is a function of **both** the model parameters θ & the density on z, $q(z|x)$
- $q(z|x)$: **variational Latent posterior**

$$\theta^* = \arg\max_{\theta} L(\theta) \qquad : \text{ original max likelihood problem}$$

p47 ELBO maximization + GMM example

$$\theta^* = \arg\max_{\theta,q} J_\theta(q) \qquad : \text{ Approximate with ELBO.}$$

The **fundamental problem** solved in any latent var. generative model is given below: **ELBO maximization** — find θ & q that maximizes the ELBO, $J_\theta(q)$.

$$\boxed{\theta^*, q^* = \arg\max_{\theta,q} \mathbb{E}_{q(z|x)} \log \frac{P_\theta(x,z)}{q(z|x)}}$$

Example from Classical ML: Gaussian Mixture Model (GMM)

z is discrete, $z \in \{1, 2, \ldots, M\}$

$$P_\theta(x) = \sum_z P_\theta(x,z) = \sum_z P_\theta(z) \cdot P_\theta(x|z)$$

$$P_\theta(x) = \sum_{j=1}^{M} P_\theta(z=j) \cdot P_\theta(x|z=j)$$

In a GMM:

$$P_\theta(z=j) = \alpha_j, \qquad P_\theta(x|z=j) = \mathcal{N}(x; \mu_j, \Sigma_j)$$

p48 GMM parameters and the ELBO optimization goal

$$P_\theta(x) = \sum_{j=1}^{M} \alpha_j \cdot \mathcal{N}(x; \mu_j, \Sigma_j) \qquad : \text{GMM}$$

parameters of a GMM:

$$\theta = \{\alpha_1, \alpha_2, \ldots, \alpha_M,\ \mu_1, \mu_2, \ldots, \mu_M,\ \Sigma_1, \Sigma_2, \ldots, \Sigma_M\}$$

since $x \in \mathbb{R}^d$, $\mu_j \in \mathbb{R}^d$, $\Sigma_j \in \mathbb{R}^{d \times d}$

$$0 \le \alpha_j \le 1, \qquad \sum_{j=1}^{M} \alpha_j = 1$$

Goal: Estimate θ, via ELBO optimization.

$$\theta^*, q^* = \arg\max_{\theta,q} J_\theta(q)$$

The above optimization can be solved **iteratively** by optimizing for θ & q, one after the other.

Algorithm: Let θ_t & q_t represent the estimate, at iteration t.

p49 Expectation Maximization (EM)

$$\text{For } t = 1 \text{ to } T: \qquad q^*_{t+1} = \arg\max_{q} J_{\theta_t}(q), \quad \text{with } \theta^t \text{ a const.}$$

$$\theta^*_{t+1} = \arg\max_{\theta} J_\theta(q_{t+1}), \quad \text{with } q_{t+1} \text{ a const.}$$

This is the **Expectation Maximization (EM) algorithm**. Can be shown that EM ensures that

$$L(\theta_{t+1}) \ \ge \ L(\theta_t)$$

EM for GMM

$$q^*_{t+1} = \arg\max_q J_\theta(q) \;=\; P_\theta(z|x)$$

It can be analytically shown that the best $q^*(z|x)$ that optimizes ELBO is $P_\theta(z|x)$.

$$q^*_{t+1} = P_{\theta_t}(z|x) = \frac{P_{\theta_t}(x|z)\cdot P_{\theta_t}(z)}{P_{\theta_t}(x)}, \qquad P_{\theta_t}(x) = \sum_z P_{\theta_t}(x|z)P_{\theta_t}(z)$$

p50 EM for GMM (M-step) and where EM fails

$$= \frac{\mathcal{N}\left(x;\mu_j^t,\Sigma_j^t\right)\cdot\alpha_j^t}{\sum_{j=1}^M \mathcal{N}\left(x;\mu_j^t,\Sigma_j^t\right)\cdot\alpha_j^t}$$

$$\theta^{t+1} = \arg\max_\theta J_\theta\left(q^*_{t+1}\right)$$

The above is solved by differentiating $J_\theta(q^*_{t+1})$ with respect to θ (α, μ, Σ) and solving for θ.

ELBO can be optimized for a Lat. variable model using the EM algorithm, **provided** $P_\theta(z|x)$ **can be computed**.

Suppose for a lat. var. model $P_\theta(z|x)$ **can't** be estimated, then **EM fails**.

Question: How to learn a lat. var. model for cases where $P_\theta(z|x)$ is **unknown**?

p51 Variational Auto-Encoders (VAE)

Latent variable Models: Given $D = \{x_i\}_{i=1}^n \sim$ iid P_x, $x_i \in \mathbb{R}^d$

$$\text{Lat. var. Model } P_\theta(x) = \int_z P_\theta(x,z)\,dz, \qquad z \in \mathbb{R}^k, \quad k \ll d$$

$$\theta^* = \arg\min_\theta D_{KL}(P_x \| P_\theta) = \arg\max_\theta \mathbb{E}_{P_x} \log P_\theta(x) \qquad : \textbf{Intractable}$$

Alternate problem using a **lower bound** on $\log P_\theta(x)$:

$$\theta^* = \arg\max_\theta J_\theta(q), \qquad J_\theta(q) = \mathbb{E}_{q(z|x)} \log \frac{P_\theta(x,z)}{q(z|x)} \;\le\; \log P_\theta(x)$$

Goal of a Neural Lat. var. Model: 1. Learn a lat. var. Model with **unknown** $P_\theta(z|x)$. 2. Enable sampling / generation from $P_\theta(x|z)$. 3. Enable posterior Inference — compute / estimate $q(z|x)$.

p52 The VAE objective (ELBO decomposed)

$$P_\theta(x) = \int_z P_\theta(x,z)\,dz, \qquad \theta^*, q^* = \arg\max_{\theta,q} J_\theta(q)$$

$$\text{Consider } J_\theta(q) = \mathbb{E}_{q(z|x)} \log \frac{P_\theta(x,z)}{q(z|x)} = \mathbb{E}_{q(z|x)} \log \frac{P_\theta(x|z)\cdot P_\theta(z)}{q(z|x)}$$

$$= \mathbb{E}_{q(z|x)} \log P_\theta(x|z) \;-\; \mathbb{E}_{q(z|x)} \log \frac{q(z|x)}{P_\theta(z)}$$

$$\boxed{J_\theta(q) = \mathbb{E}_{q(z|x)} \log P_\theta(x|z) \;-\; D_{KL}(q(z|x) \,\|\, P_\theta(z))}$$

To optimize $J_\theta(q)$ using Neural Networks

Represent $P_\theta(x|z)$ & $q(z|x)$ via Neural Networks. - $P_\theta(x|z)$: **conditional data likelihood** - $q(z|x)$: **variational Latent posterior density**

p53 How to represent probability distributions via Neural Networks?

(a) Deterministic representation. - GAN generator: $z \sim \mathcal{N}(0, I) \rightarrow \boxed{g_\theta(z)} \rightarrow \hat{x} \sim P_\theta(\hat{x})$ — **outputs the samples from the dist. that it models** - classifier: $x \rightarrow \boxed{\text{classifier}} \rightarrow y|x \sim P(y|x)$

(b) Probabilistic representation.

$$x \rightarrow \boxed{q_\varphi(z|x)} \rightarrow \text{parameters of } q(z|x)$$

outputs the parameters of the distribution that it models, but not the samples from it.

In a variational Auto-Encoder, the distributions $q(z|x)$ & $P(x|z)$ are represented using **probabilistic NNs**.

p54 VAE architecture (Encoder / Decoder)

$$\text{Given } D = \{x_i\}_{i=1}^n \sim \text{iid } P_x, \qquad P_\theta(x) = \int_z P_\theta(x, z)\, dz \quad : \text{Model.}$$

The following objective is optimized:

$$J_\theta(q) = \mathbb{E}_{q(z|x)} \log P_\theta(x|z) - D_{KL}(q(z|x) \,\|\, P_\theta(z))$$

$q(z|x)$ & $P_\theta(x|z)$ are represented using prob. NNs.

> **DIAGRAM** x → [q_ϕ(z|x)] (Encoder) → outputs params. of q_ϕ(z|x); sample z~ q_ϕ(z|x) → [P_θ(x|z)] (Decoder) → outputs params of P_θ(x|z)

There is **no direct connection** between the Enc & Dec. φ & θ denote the weights of Enc & Dec, respectively.

$$\varphi^*, \theta^* = \arg\max_{\theta,\varphi} J_\theta(q_\varphi)$$

$$\varphi^{t+1} \leftarrow \varphi^t + \alpha \nabla_\varphi J_\theta(q_\varphi), \qquad \theta^{t+1} \leftarrow \theta^t + \alpha \nabla_\theta J_\theta(q_\varphi)$$

\text{Gradient descent (ascent) to learn the Enc & Dec.}

p55 The gradient problem

$$J_\theta(q_\varphi) = \mathbb{E}_{q_\varphi(z|x)} \log P_\theta(x|z) - D_{KL}(q_\varphi(z|x) \,\|\, P_\theta(z))$$

To compute the gradients of $J_\theta(q_\varphi)$ w.r.t. φ

$$\nabla_\varphi \, \mathbb{E}_{q_\varphi(z|x)} \log P_\theta(x|z), \qquad z \sim q_\varphi(z|x)$$

The above term is similar to

$$\nabla_\psi \, \mathbb{E}_{P_\psi(v)} f_\psi(v) \qquad \text{with} \quad P_\psi(v) = q_\varphi(z|x), \;\; f_\psi(v) = \log P_\theta(x|z), \;\; v \leftrightarrow z, \; \psi \leftrightarrow \varphi$$

$$\nabla_\psi \mathbb{E}_{P_\psi(v)} f_\psi(v) = \nabla_\psi \int_v P_\psi(v) f_\psi(v)\, dv = \int_v \nabla_\psi \left(P_\psi(v) \cdot f_\psi(v)\right) dv$$

$$= \underbrace{\int_v (\nabla_\psi f_\psi(v)) \cdot P_\psi(v)\, dv}_{= \mathbb{E}_{P_\psi(v)} \nabla_\psi f_\psi(v)} + \underbrace{\int_v (\nabla_\psi P_\psi(v))\, f_\psi(v)\, dv}_{\text{is not an expectation } \therefore \text{ can't be computed}}$$

$$\Rightarrow \nabla_\varphi \, \mathbb{E}_{q_\varphi(z|x)} \log P_\theta(x|z) \quad \text{can't be computed}, \qquad z \sim q_\varphi(z|x)$$

p56 The solution: Reparameterization Trick

Recall $\nabla_\psi \mathbb{E}_{P_\psi(v)} f_\psi(v)$ cannot be computed, $v \sim P_\psi(v)$.

Express the distribution $P_\psi(v)$ in terms of another distribution (another auxiliary Rv) which is independent of the parameters ψ.

Suppose there exists an auxiliary Rv, $\epsilon \sim P_\epsilon$; P_ϵ is **not** dependent on ψ.

$$v = g(\epsilon), \qquad \text{then} \qquad \mathbb{E}_{P_\psi(v)} f_\psi(v) = \mathbb{E}_{P_\epsilon} f_\psi(g(\epsilon)) \quad : \text{Law of the Unconscious Statistician}$$

$$\nabla_\psi \mathbb{E}_{P_\psi(v)} f_\psi(v) = \nabla_\psi \mathbb{E}_{P_\epsilon} f_\psi(g(\epsilon)) \approx \frac{1}{M} \sum_{j=1}^{M} \nabla_\psi f_\psi(g(\epsilon_j)), \qquad \epsilon \sim P_\epsilon, \ \ \epsilon_j \sim P_\epsilon$$

p57 Examples of Reparameterization

(a)

$$q_\varphi(z|x) = \mathcal{N}(z; \mu_\varphi(x), \Sigma_\varphi(x))$$

DIAGRAM x → [q_ϕ(z|x)] → μ_ϕ(x), Σ_ϕ(x) (the parameters of q_ϕ(z|x) for a given input x)

Let $\epsilon \sim \mathcal{N}(0, I)$, then

$$z = \mu_\varphi(x) + \Sigma_\varphi(x) \cdot \epsilon = g(\epsilon) \qquad (\mu + \Sigma\epsilon)$$

$$z \sim \mathcal{N}(z; \mu_\varphi(x), \Sigma_\varphi(x))$$

(b) Inverse CDF Method. Suppose z is a Rv & $F_z(z)$ denotes its CDF.

$$z = F_z^{-1}(\epsilon), \qquad \epsilon \sim U[0, 1]$$

$$g(\epsilon) = F_z^{-1} \ \ (\text{Inverse CDF}), \qquad \epsilon \sim \text{uniform}[0, 1]$$

p58 Reparameterized sampling in the VAE

$$x_i \in \mathbb{R}^d, \quad z_i \in \mathbb{R}^k, \quad \mu_\varphi(x) \in \mathbb{R}^k, \quad \Sigma_\varphi(x) \in \mathbb{R}^{k \times k}$$

DIAGRAM x_i → [q_ϕ(z|x)] → μ_ϕ(x_i), Σ_ϕ(x_i); (sample) → [P_θ(x|z)] → $\hat{x}$_θ(z_i)

$$\epsilon_i \sim \mathcal{N}(0, I), \qquad \boxed{z_i = \mu_\varphi(x_i) + \Sigma_\varphi(x_i) \cdot \epsilon_i} \quad (\text{accomplished using reparam.})$$

$$\text{wanted: } z_i \sim q_\varphi(z|x) = \mathcal{N}(z; \mu_\varphi(x), \Sigma_\varphi(x))$$

$$\nabla_\varphi \mathbb{E}_{q_\varphi(z|x)} \log P_\theta(x|z) = \nabla_\varphi \mathbb{E}_{P_\epsilon} \log P_\theta(x|g(\epsilon))$$

$$= \nabla_\varphi \left(\frac{1}{M} \sum_{j=1}^{M} \log P_\theta(x|g(\epsilon_j)) \right), \qquad \epsilon_1, \epsilon_2, \ldots, \epsilon_M \sim P_\epsilon$$

To compute $\log P_\theta(x|g(\epsilon_j))$**:** parameterize $P_\theta(x|z)$ via some known distribution & use the Decoder to output its parameters.

$$\text{Eg: } P_\theta(x|z) = \mathcal{N}(x; \hat{x}_\theta(z), I)$$

p59 Gaussian likelihood → MSE loss

With the above,

$$\log P_\theta(x|z) = \log\left(\frac{1}{(2\pi)^{d/2}}\exp\left[-\|x-\hat{x}_\theta(z)\|_2^2\right]\right) \;\propto\; -\|x-\hat{x}_\theta(z)\|_2^2$$

$$\Rightarrow \quad \nabla_\varphi \mathbb{E}_{q_\varphi(z|x)} \log P_\theta(x|z) = \nabla_\varphi \mathbb{E}_{P_\epsilon} \log P_\theta(x|g(\epsilon))$$

$$= \nabla_\varphi \left[\frac{1}{M}\sum_{j=1}^{M} \|x_i - \hat{x}_\theta(z_j)\|_2^2\right]$$

$$\text{where } z_j = \mu_\varphi(x_i) + \Sigma_\varphi(x_i)\cdot\epsilon_j, \qquad \epsilon_1 \ldots \epsilon_j \sim \mathcal{N}(0, I)$$

p60 Forward pass / Backward pass

DIAGRAM x_i → [$q_\phi(z|x)$] → $\mu_\phi(x_i)$, $\Sigma_\phi(x_i)$; $\varepsilon_1 \ldots \varepsilon_M \sim N(0,I)$; $z_j = \mu_\phi(x_i) + \Sigma_\phi(x_i)\varepsilon_j$, $j = 1..M$

$$\rightarrow \boxed{P_\theta(x|z)} \; (\theta \text{ is const}) \rightarrow \hat{x}_\theta(z_j) \rightarrow \frac{1}{M}\sum \|x_i - \hat{x}_\theta(z_j)\|_2^2 \rightarrow \nabla_\varphi(\cdots)$$

Forward pass: 1. Given an $x_i \in D$, pass it through the Encoder, to obtain $\mu_\varphi(x_i)$ & $\Sigma_\varphi(x_i)$ 2. sample $\epsilon_1 \ldots \epsilon_M \sim \mathcal{N}(0, I)$ **outside the NNs**. 3. compute $z_1 \ldots z_M$ via reparameterization 4. pass all of $z_1 \ldots z_j$ through the Decoder to compute $\hat{x}_\theta(z_j)$ 5. compute $\frac{1}{M}\sum_{j=1}^{M} \|x_i - \hat{x}_\theta(z_j)\|_2^2$

Backward pass: 1. compute the grad. $\nabla_\varphi\left(\frac{1}{M}\sum_{j=1}^{M} \|x_i - \hat{x}_\theta(z_j)\|_2^2\right)$ 2. Back prop through

p61 Backprop path + the KL term

Back prop through - Decoder, with **fixed** parameters θ - Reparam, w.r.t. φ - Encoder

$$\boxed{\text{Thus, we obtain the } \nabla_\varphi \mathbb{E}_{q_\varphi(z|x)} \log P_\theta(x|z)}$$

To compute $\nabla_\varphi D_{KL}\left[q_\varphi(z|x) \,\|\, P_\theta(z)\right]$

- $P_\theta(z)$: **Latent prior**, typically assumed to be $\mathcal{N}(0, I)$
- $q_\varphi(z|x) = \mathcal{N}(z; \mu_\varphi(x), \Sigma_\varphi(x))$

$$\therefore \; D_{KL}\left[q_\varphi(z|x) \,\|\, P_\theta(z)\right] = D_{KL}\left[\mathcal{N}(\mu_\varphi(x), \Sigma_\varphi(x)) \,\|\, \mathcal{N}(0, I)\right]$$

$$= -\tfrac{1}{2}\log|\Sigma_\varphi(x)| - k + \text{tr}[\Sigma_\varphi(x)] + \|\mu_\varphi(x)\|_2^2$$

$$(\text{the notes mark this as "can be estimated"})$$

p62 Encoder training step (full picture)

$$\therefore \; \nabla_\varphi D_{KL}\left[q_\varphi(z|x) \,\|\, P_\theta(z)\right] = \nabla_\varphi\left\{-\tfrac{1}{2}\log|\Sigma_\varphi(x)| - k + \text{tr}[\Sigma_\varphi(x)] + \|\mu_\varphi(x)\|_2^2\right\} = \nabla_\varphi(KL)$$

DIAGRAM x_i → [$q_\phi(z|x)$] → $\mu_\phi(x_i)$, $\Sigma_\phi(x_i)$; $\nabla_\phi(\cdots) + \nabla_\phi(KL)$; $\varepsilon_1 \ldots \varepsilon_M \sim N(0,I)$; $z_j = \ldots$

$$\rightarrow \boxed{P_\theta(x|z)} \; (\theta \text{ is const}) \rightarrow \hat{x}_\theta(z_j) \rightarrow \frac{1}{M}\sum \|x_i - \hat{x}_\theta(z_j)\|_2^2 \rightarrow \nabla_\varphi \frac{1}{M}\sum \|\cdots\|_2^2$$

$$\varphi^{t+1} \leftarrow \varphi^t + \alpha\,\nabla_\varphi J_\theta(q_\varphi) \qquad : \text{ one training step for the Encoder.}$$

p63 Training the Decoder

$$\nabla_\theta J_\theta(q_\varphi) = \nabla_\theta \left[\mathbb{E}_{q_\varphi(z|x)} \log P_\theta(x|z)\right] - \nabla_\theta \left[D_{KL}\left(q_\varphi(z|x) \,\|\, P(z)\right)\right] = 0$$

(the KL term is independent of θ, so its gradient vanishes)

To find $\nabla_\theta \left[\mathbb{E}_{q_\varphi(z|x)} \log P_\theta(x|z)\right]$

DIAGRAM x_i → [$q_\phi(z|x)$] (ϕ is a constant) → $\mu_\phi(x_i)$, $\Sigma_\phi(x_i)$;

$$\epsilon_1 \dots \epsilon_M \sim \mathcal{N}(0, I); \quad z_i = \mu_\varphi(x_i) + \Sigma_\varphi(x_i)\epsilon_j,\ j = 1..M \rightarrow \boxed{P_\theta(x|z)} \rightarrow \hat{x}_\theta(z_j)$$

$$\rightarrow \frac{1}{M}\sum \|x_i - \hat{x}_\theta(z_j)\|_2^2 \rightarrow \nabla_\theta \frac{1}{M}\sum \|\cdots\|_2^2$$

$$\theta^{t+1} \leftarrow \theta^t + \alpha\, \nabla_\theta J_\theta(q_\varphi)$$

One training step for the Decoder

p64 Recap on training of a VAE

$$\text{Given } D = \{x_i\}_{i=1}^n \sim \text{iid } P_x, \qquad x \in \mathbb{R}^d, \quad z \in \mathbb{R}^k, \quad k \ll d$$

DIAGRAM x (d-dim) → [Enc] → $q_\phi(z|x)$ → $\mu_\phi(x)$ (k-dim), $[\sigma_1 \dots \sigma_k]$ → $\Sigma_\phi(x)$;

$$z_j\ (k\text{-dim}) \rightarrow \boxed{\text{Dec}} \rightarrow P_\theta(x|z) \rightarrow \hat{x}_\theta(z)\ (d\text{-dim})$$

$$\Sigma_\varphi(x) = \text{diag}\left([\sigma_1 \dots \sigma_k]\right), \qquad \epsilon_1 \dots \epsilon_M \sim \mathcal{N}(0, I)$$

$$z_j = \mu_\varphi(x) + [\sigma_1 \dots \sigma_k]^T \cdot I \odot \epsilon_j \qquad \text{(Reparam.)}$$

Typically,

$$q_\varphi(z|x) \sim \mathcal{N}(z; \mu_\varphi(x), \Sigma_\varphi(x)), \qquad \Sigma_\varphi(x) = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_k)$$

$$J_\theta(q_\varphi) = \mathbb{E}_{q_\varphi(z|x)} \log P_\theta(x|z) - D_{KL}\left[q_\varphi(z|x) \,\|\, P(z)\right]$$

$$= \|x - \hat{x}_\theta(z)\|_2^2 - \left(-\tfrac{1}{2}\log|\Sigma_\varphi(x)| - k + \text{tr}\left(\Sigma_\varphi(x)\right) + \|\mu_\varphi(x)\|_2^2\right)$$

θ & φ are updated **alternately** via gradient descent.

p65 Inference with a trained VAE (i): sampling

(i) Data Generation or sampling Consider the following sampling procedure. $P_\theta(x) = \int_z P_\theta(x, z)\, dz$ 1. sample a $z_{new} \sim P(z) = [\mathcal{N}(0, I)]$ 2. sample a $x_{new} \sim P_\theta(x|z)$, using the **Decoder** of the VAE.

DIAGRAM (z~ $q_\phi(z|x)$ / z_new~N(0,I)) → [$P_\theta^*(x|z)$] (Dec) → $\hat{x}_\theta(z)$ = mean of $P_\theta(x|z)$

The KL term in ELBO will ensure that

$$q_\varphi(z|x) = P(z) = \mathcal{N}(0, I) \quad \forall x$$

If the VAE is well trained, $q_\varphi(z|x) = \mathcal{N}(0, I)\ \forall x$.

$$\Rightarrow \text{ sampling from } \mathcal{N}(0, I) \implies \text{sampling from } q_\varphi(z|x)$$

p66 Inference (ii): embeddings

$\hat{x}_\theta(z_{new})$ is the output of Decoder for $z_{new} \sim \mathcal{N}(0, I)$.

Either 1. use $\hat{x}_\theta(z_{new})$ as the novel generated datapoint, $\hat{x}_\theta(z_{new}) \in \mathbb{R}^d$ 2. $x_{new} \sim \mathcal{N}(\hat{x}_\theta(z_{new}), I)$: novel generated data point. $\because P_\theta(x|z) = \mathcal{N}(\hat{x}_\theta(z), I)$.

(b) VAEs for latent posterior Inference or Embedding or Feature Extraction

Consider the trained Encoder of a VAE,

DIAGRAM x_test → [q_ϕ^*(z|x)] (Encoder) → μ_ϕ(x_test), Σ_ϕ(x_test), z_test∈ℝ^k

The Embedding or the feature vector for x_{test} is given by

$$\text{a) } z_{test} = \mu_\varphi(x_{test}) \qquad \text{b) } z_{test} \sim \mathcal{N}(\mu_\varphi(x_{test}), \Sigma_\varphi(x_{test}))$$

p67 Posterior collapse in a naive VAE

DIAGRAM x → [q_ϕ(z|x)] → z → [P_θ(x|z)] → x̂_θ(z)

In a VAE, the second term in ELBO, $D_{KL}\left(q_\varphi(z|x) \parallel P(z)\right)$ is minimized.

$$\forall x,\ q_\varphi(z|x) \text{ is forcefully made to be the same as } P(z)\ (\mathcal{N}(0, I))$$

The **decoder** will see it difficult to differentiate between two input samples x_i & x_j.

$$\therefore q_\varphi(z|x_i) = q_\varphi(z|x_j) = \mathcal{N}(0, I), \qquad z_i \sim q_\varphi(z|x_i),\ z_j \sim q_\varphi(z|x_j)$$

Solution for posterior collapse in VAE

$$J_\theta(q_\varphi) = \underbrace{\|x - \hat{x}_\theta(z)\|_2^2}_{\text{reconstruction term on } x} + \underbrace{D_{KL}\left(q_\varphi(z|x) \parallel P(z)\right)}_{\text{Regularization term on } z}$$

p68 VAE as a regularized auto-encoder; β-VAE

VAE can be viewed as an auto-Encoder with a **regularized latent space** (Regularized Auto-Encoder).

Typical regularized cost in ML looks like

$$\underbrace{L_\theta(\cdot)}_{\text{Cost fn.}} + \underbrace{\alpha}_{\text{reg. const}} \cdot \underbrace{\Omega(\theta)}_{\text{regularization fn.}}$$

In a VAE, viewed as regularized AE, the **regularization constant is absent**. VAE can be made to trade-off between cost & regularization by adding a regularization constant to ELBO.

$$\hat{J}_\theta(q_\varphi) = \|x - \hat{x}_\theta(z)\|_2^2 + \beta\, D_{KL}\left[q_\varphi(z|x) \parallel P(z)\right]$$

β-VAE, $0 \le \beta \le 1$, β - hyper parameter. - higher β → lead to posterior collapse, better generation - lower β → reconstructions are better, generation will suffer $\because q_\varphi(z|x) \neq P(z)$

p69 Vector quantized VAE (VQ-VAE)

In VQ-VAE, the latent space is **discrete** & **vector-quantized** using a learnable dictionary.

$$\{z_1, z_2, \ldots, z_M\} \quad : \text{ lat. Dictionary}$$

DIAGRAM x_i → [q_ϕ] (Enc) → $\hat{z}_e(x_i)$; z_q(x_i) → [P_θ] (Dec) → $\hat{x_i}$

$$z_i \in \mathbb{R}^k, \qquad \hat{z}_e(x_i) \in \mathbb{R}^{k \times p} \quad ; \; p\text{-vectors from the lat. dict}$$

Suppose there are M latent vectors of K-dim each in the dictionary.

$$L = \{z_1, z_2, \ldots, z_M\}, \qquad z_i \in \mathbb{R}^k$$

$$z_q(x_i) = z_{j^*}, \qquad j^* = \arg\min_{j=1..M} \|z_e(x_i) - z_j\|_2^2$$

$z_e(x_i)$: column of $\hat{z}_e(x_i) \longrightarrow$ **vector quantization**

In practice, the latent space is designed to be a tensor of latent dictionary.

p70 VQ-VAE structure, training, and embedding inference

$$\hat{z}_e(x_i) = \begin{bmatrix} | & | & | & | & | & | \end{bmatrix} \qquad \text{each row is a vector from the } k \times p \text{ dictionary } L$$

Each **column** of $\hat{z}_e(x_i)$ is vector quantized.

Training a vq-VAE

$$J_\theta(q) = \|x - \hat{x}_\theta(\hat{z}_q)\|_2^2 + \|z_e(x_i) - z_q(x)\|_2^2$$

In addition to the Encoder & Decoder parameters, the **latent dictionary is also learned**.

$$z_j^{t+1} \leftarrow z_j^t - \alpha \nabla_{z_j} (J_\theta(q))$$

Inference in a vq-VAE

(a) Embedding Extraction / posterior Inference

DIAGRAM x_test → [q_ϕ^*] → $\hat{z}_e$(x_test)$\in \mathbb{R}$ ^k× p

$$\hat{z}_q(test) = \arg\min_{j=1..M} \|\hat{z}_e(x_{test}) - z_j\|_2^2$$

p71 Sampling in a vq-VAE

(b) Sampling in vq-VAE

DIAGRAM $\hat{z}$_q(x) → [P_θ^*] → $\hat{x}$_θ^*

To be able to sample from the decoder of vq-VAE, we need to **sample from the distribution of** $\hat{z}_q(\cdot)$.

To learn to sample from the dist. of $\hat{z}_q(\cdot)$, a **generative model** is fit to samples of z_q. Eg: GMM on the samples from the o/p of Encoder corresponding to all $\#p$ datapoints.

Sample from the learned dist. of z_q & use it as the input to the Decoder.

— End of VAEs —

Transcribed page-by-page from *GenAI-IITM.pdf* (handwritten notes, pages 43–71). Notation preserved as written. Built for revision — verify against the source for coursework.